

Backend Basics & Node.js Notes

1. Backend Basics

How the Web Works

When you open a website:

1. You enter a URL in the browser.
2. Browser sends a request to the server.
3. Server processes the request.
4. Server sends a response back.
5. Browser displays the webpage.

Example

Browser → Request → Server

Browser ← Response ← Server

When you visit:

<https://google.com>

Your browser sends an HTTP request to Google's server and receives HTML, CSS, JS, images, etc.

Client-Server Architecture

Client

The client is the application that requests data.

Examples:

- Chrome Browser
- Mobile Apps
- React Application

Server

The server provides data and services.

Examples:

- Node.js Server
- Express Server

Architecture

Client → Request → Server

Client ← Response ← Server

Example

Instagram App



Instagram Server



Returns Posts

Request-Response Cycle

Request

A request contains:

URL

Method

Headers

Body

Example:

GET /users

Response

A response contains:

Status Code

Headers

Data

Example:

```
{  
  "name": "Aditya"  
}
```

Flow

Browser



Request



Server



Processing



Response



Browser

2. Node.js

Introduction to Node.js

Node.js is a JavaScript runtime environment that allows JavaScript to run outside the browser.

Before Node.js:

JavaScript → Browser Only

After Node.js:

JavaScript → Browser + Server

Features

- Fast
- Event-driven
- Non-blocking I/O
- Single-threaded
- Asynchronous

Example

```
console.log("Hello Node.js");
```

Run:

```
node app.js
```

Node REPL

REPL =

Read

Eval

Print

Loop

Open terminal:

node

Example:

> 5 + 5

10

> "Aditya"

'Aditya'

Used for quick testing.

Process Object

The process object provides information about the current Node.js process.

Example

```
console.log(process.version);
```

Output:

v22.x.x

Command Line Arguments

```
console.log(process.argv);
```

Run:

```
node app.js Aditya 21
```

Output:

```
[  
  'node',  
  'app.js',  
  'Aditya',  
  '21'  
]
```

Global Objects

Available everywhere without importing.

Common Global Objects

```
console.log(__dirname);
```

```
console.log(__filename);
```

```
setTimeout()
```

```
setInterval()
```

```
clearTimeout()
```

Example

```
setTimeout(() => {  
  console.log("Hello");  
}, 2000);
```

Modules

A module is a reusable piece of code.

Benefits

- Code Reusability
 - Better Organization
 - Maintainability
-

Built-in Modules

FS Module (File System)

Used to work with files.

```
const fs = require("fs");
```

```
fs.writeFileSync("demo.txt", "Hello");
```

Path Module

Works with file paths.

```
const path = require("path");
```

```
console.log(path.basename(__filename));
```

Output:

app.js

HTTP Module

Creates servers.

```
const http = require("http");

const server = http.createServer((req, res) => {
  res.end("Hello World");
});

server.listen(3000);
```

OS Module

Provides OS information.

```
const os = require("os");

console.log(os.platform());
```

Output:

win32

Custom Modules

Create:

math.js

```
function add(a, b) {
  return a + b;
}
```

```
module.exports = add;
```

app.js

```
const add = require("./math");
```

```
console.log(add(2, 3));
```

Output:

5

npm

npm = Node Package Manager

Used to install external packages.

Check Version

```
npm -v
```

Install Package

```
npm install cowsay
```

package.json

Stores project information.

Create:

```
npm init
```

Example:

```
{  
  "name": "backend-project",  
  "version": "1.0.0"  
}
```

Contains:

- Project Name
 - Version
 - Dependencies
 - Scripts
-

Installing Packages

Local Install

```
npm install express
```

Global Install

npm install -g nodemon

Install Specific Version

npm install express@4.18.2

nodemon

Automatically restarts the server when files change.

Install

npm install -g nodemon

Run

nodemon app.js

Without nodemon:

node app.js

Every change requires restarting manually.

With nodemon:

nodemon app.js

Server restarts automatically.

3. Express.js Notes

Introduction to Express

Express.js is a fast and lightweight web framework for Node.js.

Without Express, creating servers in Node.js requires more code.

Why Express?

- Simplifies server creation
- Easy routing
- Middleware support
- REST API development
- Faster backend development

Install Express:


```
npm install express
```

Import Express:

```
const express = require("express");
```

Creating a Server

Step 1: Create Express App

```
const express = require("express");
```

```
const app = express();
```

Step 2: Start Server

```
app.listen(3000, () => {
```

```
  console.log("Server Started");
```

```
});
```

Complete Example

```
const express = require("express");
```

```
const app = express();
```

```
app.listen(3000, () => {
```

```
  console.log("Server Running");
```

```
});
```

Open:

```
http://localhost:3000
```

Routing

Routing means deciding what should happen when a user visits a particular URL.

Syntax

```
app.METHOD(PATH, CALLBACK);
```

GET Route

```
app.get("/", (req, res) => {
```

```
  res.send("Home Page");
```

```
});
```

Multiple Routes

```
app.get("/", (req, res) => {  
  res.send("Home");  
});
```

```
app.get("/about", (req, res) => {  
  res.send("About Page");  
});
```

```
app.get("/contact", (req, res) => {  
  res.send("Contact Page");  
});
```

Route Parameters

Used when values change dynamically in URLs.

Example

```
app.get("/user/:id", (req, res) => {  
  res.send(req.params.id);  
});
```

URL:

/user/101

Output:

101

Multiple Parameters

```
app.get("/user/:id/:name", (req, res) => {  
  console.log(req.params);  
});
```

URL:

/user/101/aditya

Output:

```
{
```

```
    id: "101",  
    name: "aditya"  
  }  
}
```

Access Parameters

req.params.id

req.params.name

Query Strings

Query strings send extra information through URL.

Example

/search?q=nodejs

Here:

q=nodejs

is a query parameter.

Access Query

```
app.get("/search", (req, res) => {  
  res.send(req.query.q);  
});
```

URL:

/search?q=express

Output:

express

Multiple Queries

URL:

/search?q=node&page=2

Output:

```
{  
  q: "node",  
  page: "2"  
}
```

Access:

req.query

Sending Responses

Express provides multiple methods for sending responses.

res.send()

Sends text, HTML, objects, arrays.

```
app.get("/", (req, res) => {  
  res.send("Hello World");  
});
```

Send HTML

```
app.get("/", (req, res) => {  
  res.send("<h1>Hello</h1>");  
});
```

Send Object

```
app.get("/", (req, res) => {  
  res.send({  
    name: "Aditya",  
    age: 21  
  });  
});
```

Express automatically converts it to JSON.

res.json()

```
app.get("/", (req, res) => {  
  res.json({  
    success: true  
  });  
});
```

```
});
```

Output:

```
{  
  "success": true  
}
```

Express Middleware

Middleware is a function that executes between request and response.

Flow

Request



Middleware



Route



Response

Syntax

(req, res, next)

- req → Request Object
 - res → Response Object
 - next → Pass control to next middleware
-

Example

```
app.use((req, res, next) => {  
  console.log("Middleware Executed");  
  next();  
});
```

Every request passes through this middleware.

Custom Middleware

Middleware created by developers.

Example

```
function logger(req, res, next) {  
  console.log(req.method);  
  next();  
}
```

```
app.use(logger);
```

Output:

GET

POST

PUT

depending on request type.

Authentication Middleware

```
function auth(req, res, next) {  
  const isLoggedIn = true;  
  
  if (isLoggedIn) {  
    next();  
  } else {  
    res.send("Unauthorized");  
  }  
}
```

Use:

```
app.get("/profile", auth, (req, res) => {  
  res.send("Profile Page");  
});
```

Third-Party Middleware

Middleware provided by external packages.

Install package:

npm install morgan

Morgan Middleware

```
const morgan = require("morgan");
```

```
app.use(morgan("dev"));
```

Logs request information.

Example Output:

GET / 200

POST /login 200

Express Built-in Middleware

JSON Middleware

```
app.use(express.json());
```

Converts JSON request body into JavaScript object.

URL Encoded Middleware

```
app.use(express.urlencoded({ extended: true }));
```

Used for form data.

Request Lifecycle

Client Request

↓

Middleware 1

↓

Middleware 2

↓

Route Handler

↓

Response
